

Unit - IV

INTERMEDIATE SQL

FOREIGN KEY

A foreign key is a column or a set of columns in one table that uniquely references the primary key or unique key in another table, establishing a link between them.

Example Structure

- **Parent Table (Customers):** Contains customer_id as the [Primary Key](#).
- **Child Table (Orders):** Contains order_id as its own primary key and customer_id as the [Foreign Key](#) pointing to the Customers table

FOREIGN KEY Constraints

MySQL supports foreign keys, which permit cross-referencing related data across tables, and foreign key constraints, which help keep the related data consistent.

A foreign key relationship involves a parent table that holds the initial column values, and a child table with column values that reference the parent column values. A foreign key constraint is defined on the child table.

The essential syntax for defining a foreign key constraint in a CREATE TABLE or ALTER TABLE statement includes the following:

```
[CONSTRAINT [symbol]] FOREIGN KEY  
  [index_name] (col_name, ...)  
  REFERENCES tbl_name (col_name,...)  
  [ON DELETE reference_option]  
  [ON UPDATE reference_option]
```

reference_option:

```
RESTRICT | CASCADE | SET NULL | NO ACTION | SET DEFAULT
```

Transactions

Transactions are the core function of maintaining data consistency in the SQL world. Basically, all mainstream relational databases support the transaction feature, and even some non-relational databases software can support the transaction feature (e.g. Mongo DB -Transaction). You can imagine its importance in practical systems.

Transactions are important because there are some statements that we want to completely succeed or to completely fail, but a partial success is not an option. An example is

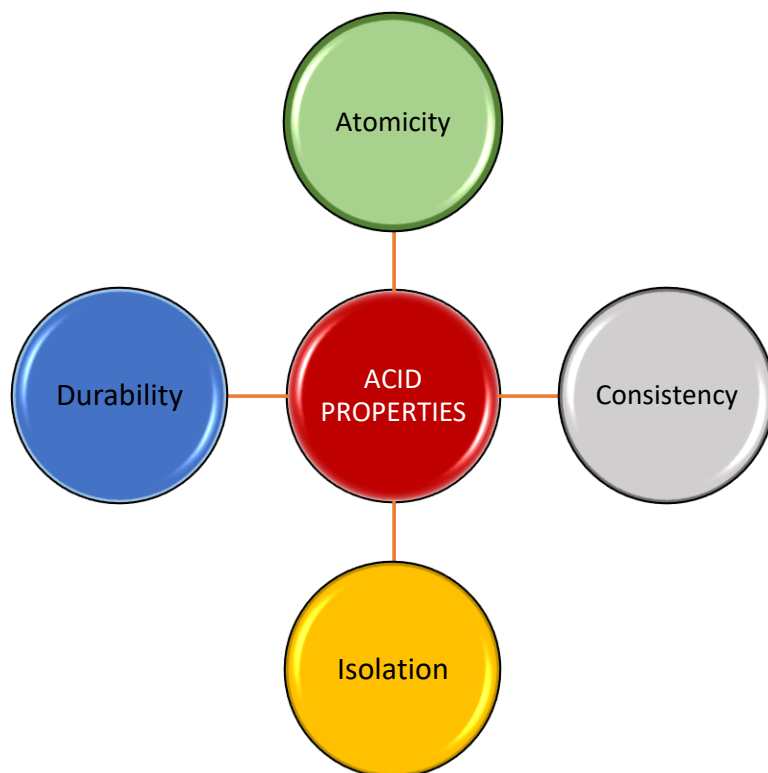
transferring money to the bank.

Transaction Control Language (TCL) Commands

You control the boundaries of an explicit transaction using specific TCL commands:

Command	Action
BEGIN TRANSACTION	Explicitly marks the starting point of a new transaction block.
COMMIT	Saves all modifications permanently to the database and ends the transaction.
ROLLBACK	Erases all modifications made since the start of the transaction and reverts data to its initial state.
SAVEPOINT	Sets a tactical checkpoint within the transaction, allowing a partial rollback without losing everything.

The ACID Properties



Atomicity:

The Smallest Unit. If we want to use transaction, then we should regard a group of queries as the smallest execution unit in SQL. That is to say, even if there are multiple statements in a transaction, we have to treat them as one. The reason is if a bug is triggered when we are half way through a group of queries for updating anything, the whole batch of updates should be discarded. Through the design of atomicity, the database can ensure that SQL statements are completely bound together and will not be split. Otherwise, it would be terrible if a crash event caused a large transaction to fail halfway through execution, and the execution result of SQL was cut into two parts, where the first half was executed, but the second half wasn't.

Example

Take the case of this chapter as an example when we want to perform a bank transaction between accounts A and B. Without atomicity, it is possible that after the database finishes the deduction of account A, it crashes without handling the money transfer to B. In this case, it is a disaster because A has been debited, but B will not receive the money. That's because, without atomicity implemented by the database, the execution results of the withdrawal and deposit statements are split and not treated as a complete set.

Consistency:

Always Make Data Consistent. In the database, we will set many diverse constraints. In the case of many transaction operation successes and failures, a database with the ability to remain consistent can ensure that these constraints are satisfied. Therefore, consistency focuses on a key point, keep the data in the correct appearance. We learned how to use the ROLLBACK keyword in the previous chapter to ensure the correctness of the data and return it to the correct state (before the transaction started). Consistency helps us to ensure that data remains in the correct state.

Example

In the case of this chapter, with consistency, we can use the ROLLBACK keyword to go back to the original if there is any crash event that happens during the execution.

Isolation: Transactions:

Do Not Affect Each Other. Remember we mentioned at the beginning that the concept of ACID is a must-have basic knowledge to undertake systems with a large amount of traffic? Isolation is used to solve data problems when traffic increases; it ensures that transactions do not affect each other. When many transactions occur, some transactions that are not exempted may need to modify the same record, which will make the problem become very complicated, and it will be a challenge to determine which version is correct. Isolation requires the database to ensure that transactions will not affect each other to a certain extent. To understand to what extent, you can refer to Isolation Levels. The implementation is similar across different databases.

Example

Consider a public account A where different people can withdraw money from A at the same time. Isolation can ensure that the data columns that are required for both are locked by one first, and the other transaction is queued until the record has been unlocked. But be careful, an inappropriate lock mechanism may cause a deadlock error and cause the database to crash, so still pay attention to the possibility of related problems.

Durability:

Always Exist. In addition to software crashes, there may also be hardware-level failures such as power outages in the real world. For such events, a database with durability capability can ensure that as long as a result has been committed, it must exist in the hard disk data permanently. This is a relatively simple concept, but it is vital that we ensure the data will not be lost.

Example

A temporary power outage in the data centre, where our transaction database is stored resulted in a few minutes of application downtime. After recovery, we find many data that should have executed "commit" are lost. This is an example of the problem of no durability.

Social Media Analytics – Complete SQL Script

Database Design Overview:

Entities

- Users
- Platforms (Instagram, X, LinkedIn, etc.)
- Posts
- Reactions (Likes, Comments, Shares)
- Followers

Table Creation Script (DDL)

1. Users Table

```
CREATE TABLE users (  
    user_id INT PRIMARY KEY,  
    username VARCHAR(50) NOT NULL,  
    join_date DATE,  
    country VARCHAR(50)  
);
```

2. Platforms Table

```
CREATE TABLE platforms (  
    platform_id INT PRIMARY KEY,  
    platform_name VARCHAR(30) UNIQUE NOT NULL  
);
```

3. Posts Table

```
CREATE TABLE posts (  
    post_id INT PRIMARY KEY,  
    user_id INT,  
  
    platform_id INT,
```

```
post_date DATE,  
content_type VARCHAR(20), -- image, video, reel, text  
CONSTRAINT fk_post_user FOREIGN KEY (user_id)  
REFERENCES users(user_id),  
  
CONSTRAINT fk_post_platform FOREIGN KEY (platform_id)  
REFERENCES  
platforms(platform_id)  
);
```

4. Reactions Table

```
CREATE TABLE reactions (  
reaction_id INT PRIMARY KEY,  
post_id INT,  
reaction_type VARCHAR(20), -- like, comment, share  
reaction_date DATE,  
  
CONSTRAINT fk_reaction_post FOREIGN KEY (post_id)  
REFERENCES posts(post_id)  
);
```

5. Followers Table

```
CREATE TABLE followers (  
follower_id INT,  
following_id INT,  
follow_date DATE,  
  
PRIMARY KEY (follower_id, following_id),  
  
CONSTRAINT fk_follower FOREIGN KEY (follower_id)  
REFERENCES users(user_id),  
  
CONSTRAINT fk_following FOREIGN KEY (following_id)  
REFERENCES users(user_id)  
);
```

Sample Data Insertion (DML)

Users

```
INSERT INTO users VALUES (1, 'arjun_ai', DATE '2024-01-10', 'India');  
INSERT INTO users VALUES (2, 'meera_data', DATE '2024-02-15', 'India');  
INSERT INTO users VALUES (3, 'john_ml', DATE '2023-11-05', 'USA');  
INSERT INTO users VALUES (4, 'sara_cloud', DATE '2024-03-01', 'UK');
```

Platforms

```
INSERT INTO platforms VALUES (1, 'Instagram');  
INSERT INTO platforms VALUES (2, 'X');  
INSERT INTO platforms VALUES (3, 'LinkedIn');
```

Posts

```
INSERT INTO posts VALUES (101, 1, 1, DATE '2024-06-01', 'reel');  
INSERT INTO posts VALUES (102, 2, 1, DATE '2024-06-02', 'image');  
INSERT INTO posts VALUES (103, 3, 2, DATE '2024-06-03', 'text');  
INSERT INTO posts VALUES (104, 1, 3, DATE '2024-06-04', 'article');
```

Reactions

```
INSERT INTO reactions VALUES (1001, 101, 'like', DATE '2024-06-01');  
INSERT INTO reactions VALUES (1002, 101, 'comment', DATE '2024-06-01');  
INSERT INTO reactions VALUES (1003, 101, 'share', DATE '2024-06-02');  
INSERT INTO reactions VALUES (1004, 102, 'like', DATE '2024-06-02');  
INSERT INTO reactions VALUES (1005, 103, 'like', DATE '2024-06-03');  
INSERT INTO reactions VALUES (1006, 104, 'comment', DATE '2024-06-04');
```

Followers

```
INSERT INTO followers VALUES (2, 1, DATE '2024-05-01');  
INSERT INTO followers VALUES (3, 1, DATE '2024-05-02');  
INSERT INTO followers VALUES (4, 2, DATE '2024-05-03');
```

1. Total Engagement per Post

```
SELECT  
p.post_id,  
COUNT(r.reaction_id) AS total_engagement  
FROM posts p  
LEFT JOIN reactions r ON p.post_id = r.post_id  
GROUP BY p.post_id;
```

post_id	total_engagement
101	3
102	1
103	1
104	1

2. Engagement Breakdown (Like / Comment / Share)

```
SELECT  
p.post_id,  
r.reaction_type,  
COUNT(*) AS reaction_count  
FROM posts p  
JOIN reactions r ON p.post_id = r.post_id  
GROUP BY p.post_id, r.reaction_type;
```

post_id	reaction_type	reaction_count
101	like	1
101	comment	1
101	share	1
104	comment	1
102	like	1
103	like	1

3. Top 5 Most Engaging Posts

```
SELECT *  
FROM (  
  SELECT  
    p.post_id,  
    COUNT(r.reaction_id) AS engagement,  
    RANK() OVER (ORDER BY COUNT(r.reaction_id) DESC) AS rnk  
  FROM posts p  
  LEFT JOIN reactions r ON p.post_id = r.post_id  
  GROUP BY p.post_id  
)  
WHERE rnk <= 5;
```

post_id	engagement	rnk
101	3	1
104	1	2
102	1	2
103	1	2

4. Most Engaging Users

```
SELECT  
u.username,  
COUNT(r.reaction_id) AS total_engagement  
FROM users u  
JOIN posts p ON u.user_id = p.user_id  
LEFT JOIN reactions r ON p.post_id = r.post_id  
GROUP BY u.username  
ORDER BY total_engagement DESC;
```

username	total_engagement
arjun_ai	4
meera_data	1
john_ml	1

5. Users Who Never Posted

```
SELECT u.username
FROM users u
LEFT JOIN posts p ON u.user_id = p.user_id
WHERE p.post_id IS NULL;
```

username
sara_cloud

6. Average Reactions per Post

```
SELECT
AVG(reaction_count) AS avg_reactions
FROM (
SELECT post_id, COUNT(*) AS reaction_count
FROM reactions
GROUP BY post_id
);
```

avg_reactions
1.5000

7. Platform-wise Engagement

```
SELECT
pl.platform_name,
COUNT(r.reaction_id) AS total_engagement
FROM platforms pl
JOIN posts p ON pl.platform_id = p.platform_id
LEFT JOIN reactions r ON p.post_id = r.post_id
GROUP BY pl.platform_name;
```

platform_name	total_engagement
Instagram	4
LinkedIn	1
X	1

8. Peak Posting Day

```
SELECT
post_date,
COUNT(*) AS post_count
FROM posts
GROUP BY post_date
ORDER BY post_count DESC;
```

post_date	post_count
2024-06-01	1
2024-06-02	1
2024-06-03	1
2024-06-04	1

9. Users with More Than 1,000 Engagements (Scalable Query)

```
SELECT
u.username
FROM users u
JOIN posts p ON u.user_id = p.user_id
JOIN reactions r ON p.post_id = r.post_id
GROUP BY u.username
HAVING COUNT(r.reaction_id) > 1000;
```

username

10. Followers Count per User

```
SELECT
u.username,
COUNT(f.follower_id) AS follower_count
FROM users u
LEFT JOIN followers f ON u.user_id = f.following_id
GROUP BY u.username;
```

username	follower_count
arjun_ai	2
meera_data	1
john_ml	0
sara_cloud	0